Fall 2026

CS-UY 2413 (Sections A and B): Design and Analysis of Algorithms

Prof. Lisa Hellerstein
lisa.hellerstein@nyu.edu

Slides include material from Introduction to Algorithms, 4th Edition, MIT Press by Cormen, Leiserson, Rivest, and Stein.

- Recommended reading: Chap. 7 Quicksort
  Chap. 8 Sorting in Linear Time
  Chap. 5 Randomized Algorithms
- Review of Probability
  Probability Handout on EdStem
  Appendix C (Counting and Probability)

Lecture 10

Counting pairs

- Set S = {1,2,3,…, n} e.g., n=3, S={1,2,3}
- Number of unordered pairs of distinct elements of S

$$\frac{n(n-1)}{2}$$

1,2 1,3 2,3

- Number of unordered pairs of elements of S, elements could be equal)

$$\frac{n(n+1)}{2}$$

1,1 1,2 1,3 2,2 2,3 3,3

$\frac{n(n-1)}{2} + n$

1,1 2,2 3,3 . . . n,n

Analysis of Randomized Quicksort

Idea

- Compute expected number of comparisons (between array elements) performed by Randomized Quicksort
- Use indicator random variables!
- Initial array $A = [z_1, z_2, z_3, \ldots, z_n]$
- For each pair of elements in the array, z_i and z_j where $i \neq j$, define indicator random variable X_{ij}
  - $X_{ij} = 1$ if z_i is compared to z_j
    $= 0$ if not
- Let X be a random variable that is the total number of comparisons performed by Randomized Quicksort

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}$$

- $E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} E[X_{ij}]$

- Now have the problem:
  What is $E[X_{ij}]$?
  Equivalently, what is the probability that z_i is compared to z_j?

- Easy to answer this question if z_i is the minimum element in the array, and z_j is the maximum element.
- why?

$$E[X_{ij}] = \text{Prob}[\text{Pivot is not min or max}] \cdot 0 + \text{Prob}[\text{Pivot is min or max}] \cdot 1 = \frac{2}{n}$$

- What about for general z_i and z_j?

Randomized Quicksort: Expected Number of Comprisons

- Call the elements in the input array $z_1, z_2, \ldots, z_n$ and number them so that
  $$z_1 < z_2 < \ldots < z_n$$



- Let $Z_{ij} = \{z_i, z_{i+1}, \ldots, z_j\}$
- Lemma: During the execution of Randomized-Quicksort, an element z_i is compared to an element z_j, where $i < j$, iff the first time that Randomized-Quicksort chooses an element in Z_{ij} as a pivot, it chooses either z_i or z_j.

$i=5$
$j=9$

$z_5 < z_6 < z_7 < z_8 < z_9$

$Z_{ij} = \{z_5, z_6, z_7, z_8, z_9\}$

Randomized Quicksort: Expected Number of Comprisons

- Call the elements in the input array $z_1, z_2, \ldots, z_n$ and number them so that
$$z_1 < z_2 < \ldots < z_n$$
- Let $Z_{ij} = \{z_i, z_{i+1}, \ldots, z_j\}$
- Lemma: During the execution of Randomized-Quicksort, an element z_i is compared to an element z_j, where $i < j$, iff the first time that Randomized-Quicksort chooses an element in Z_{ij} as a pivot, it chooses either z_i or z_j .
- Pf:
  - Consider execution of Randomized Quicksort
  - Consider set of elements being sorted in each recursive call
  - The elements of Z_{ij} stay together in the same set until some element in Z_{ij} is chosen as a pivot
  - If z_i or z_j is chosen as pivot, then comparison between z_i and z_j is performed. If another element in Z_{ij} is chosen, then Partition puts z_i and z_j into different sets, and they are never compared.

Randomized Quicksort: Expected Number of Comprisons

- Call the elements in the input array $z_1, z_2, \ldots, z_n$ and number them so that
$$z_1 < z_2 < \ldots < z_n$$
- Let $Z_{ij} = \{z_i, z_{i+1}, \ldots, z_j\}$
- **Lemma:** During the execution of Randomized-Quicksort, if z_i and z_j are two elements with $i < j$, then the probability that a comparison is performed between z_i and z_j is $\dfrac{2}{j - i + 1}$
- **Pf:**
  - Consider execution of Randomized-Quicksort
  - The elements of Z_{ij} stay together in the same set until some element in Z_{ij} is chosen as the pivot
  - Consider first time the chosen pivot is in Z_{ij}.
  - Given that some element in Z_{ij} was chosen, it was equally likely to be any of the elements in Z_{ij}.
  - Since there are $j - i + 1$ elements in Z_{ij}, the probability that z_i or z_j was chosen was $\dfrac{2}{j - i + 1}$.
  - If z_i or z_j was chosen as pivot, then comparison between z_i and z_j was performed. If another element in Z_{ij} was chosen, then comparison between z_i and z_j never performed.

Randomized Quicksort: Expected Number of Comprisons

- Lemma says: during the execution of Randomized-Quicksort, if z_i and z_j are two elements with $i < j$, then the probability that a comparison is performed between z_i and z_j is $\frac{2}{j-i+1}$
- **Theorem:** Expected number of comparisons performed by Randomized Quicksort is $O(n \log n)$
- **Proof:** For all $i < j, (1 \leq i, j \leq n)$ let X_{ij} be a random variable whose value is 1 if a comparison is performed betwen z_i and z_j.
  - Let X be random variable, equal to the total number of comparisons performed by run of Randomized Quicksort. Then

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}$$

  - E[X] is expected number of comparisons performed by Randomized Quicksort
  - We will upper bound E[X]

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}$$

so $E[X] = E[\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} E[X_{ij}]$ by linearity of expectation

$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} P[X_{ij} = 1]$$

$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j-i+1}$$ by previous lemma

$$= \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1}$$ change of vars, $k = j - i$

$\sum_{i=1}^{n} [\sum_{k=1}^{n} \frac{2}{k}]$ = $n * \sum_{k=1}^{n} \frac{2}{k}$ b/c $i-1 \leq n$

$$= 2n \sum_{k=1}^{n} \frac{1}{k}$$

$= 2n(1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \cdots + \frac{1}{n})$

$= 2n * \Theta(\log n)$ $= \Theta(n \log n)$

- ⇒ Expected number of comparisons performed by Randomized Quicksort is $O(n \log n)$
  ⇒ Expected runtime of Randomized Quicksort is $O(n \log n)$
- Can also show it is $\Omega(n \log n)$ based on previous analysis
  ⇒ Expected runtime of Randomized Quicksort is $\theta(n \log n)$
- Always!
  - Best-case expected runtime
  - Worst-case expected runtime

$\theta(n \log n)$
$\theta(n \log n)$

Two Facts

- Fact 1: $\log_2(n!) = \theta(n \log n)$
- Proof:
  - To show $\log_2(n!) = O(n \log n)$ we show
    $\log_2(n!) = \log_2(1 \cdot 2 \cdot 3 \cdot \ldots \cdot n) \leq \log_2(n^n) = n \log_2 n$
  - To show $\log_2(n!) = \Omega(n \log n)$ we show
    $\log_2(n!) \geq \log_2(\frac{n}{2} \cdot (\frac{n}{2} + 1) \cdot \ldots \cdot n) \geq \log_2((n/2)^{n/2})$

$$= \frac{n}{2} \log_2 \frac{n}{2} = \frac{n}{2}(\log_2 n - 1) = \frac{1}{2}(n \log_2 n - n)$$

- Fact 2: Any binary tree with n leaves (not necessarily complete) must have height at least $\log_2 n$
  - Pf: Let h be height of a binary tree with n leaves
    Since tree is binary, it has at most 2^h leaves
    $n \leq 2^h \Rightarrow \log_2 n \leq h$

Lower Bound on Comparison Sorting

Comparison Sort Algorithm

- A comparison sort algorithm works by repeatedly performing comparisons between array elements
  - e.g., if A[j] > A[i]
  - or x=A[i]
if x > A[j] then
- A comparison sort algorithm doesn't compare array elements to constants, or use array elements as indices. So e.g., it can't do
  - if A[j] > 10 then ….
  - C[A[j]] = …. Where C is another array
- All the sorting algorithms we've studied so far have been comparison sort algorithms

Warm-Up

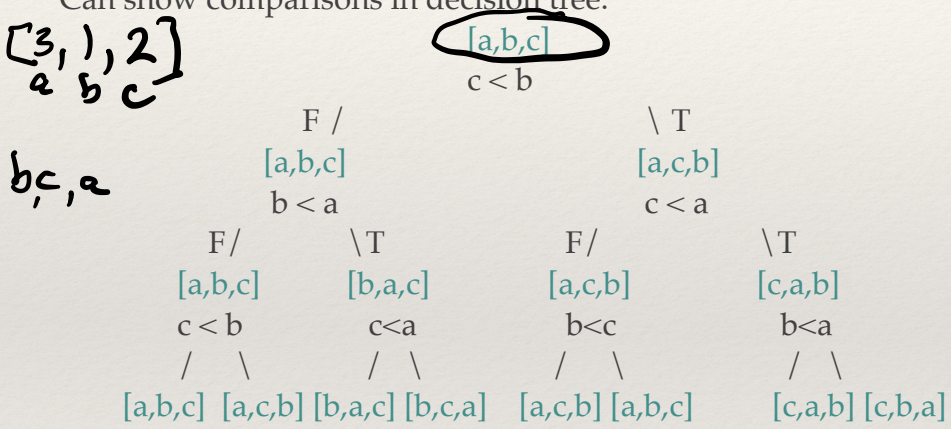
- Suppose Alg is a comparison sort algorithm
  - That's all you know about it
- Claim: The (worst-case) number of comparisons performed by Alg on an array of length n is $\Omega(n)$
- Proof of Claim?

Every elt must be involved in at least 1 comparison
Every comparison involves 2 elements
$\Rightarrow \geq \frac{n}{2}$ comparisons
$\Rightarrow$ # comparisons is $\Omega(n)$

- Just showed:
  - The worst-case number of comparisons performed by any comparison sort algorithm is $\Omega(n)$
  - $\Omega(n)$ is a lower bound on the running time of any comparison sort algorithm
- Better lower bound on number of comparisons?

Recall Bubblesort

- Which comparisons are performed on array [a,b,c]?
  Depends on relative values of a,b,c.
  Can show comparisons in decision tree:




```
BUBBLESORT(A, n)
1  for i = 1 to n − 1
2      for j = n downto i + 1
3          if A[j] < A[j − 1]
4              exchange A[j] with A[j − 1]
```

Leaves show output array (each is a permutation of elements in the original array)

- Could draw similar tree for MergeSort, HeapSort,…

- Theorem: Worst-case number of comparisons performed by any comparison sort algorithm is $\Omega(n \log n)$

Proof of $\Omega(n \log n)$ lower bound on comparison sorting

- Let Alg be a comparison sort algorithm
- Consider using Alg to sort an array A of length n
- Consider associated decision tree showing comparisons performed by the algorithm
  - Binary tree
  - Each node has a comparison between two elements of the input array
  - Each leaf of the decision tree has a permutation of elements of A
- Worst-case number of comparisons performed by Alg = height of tree
- There must be one leaf for every possible permutation of elements of A
  - n! possible permutations
  - Tree has at least n! leaves
  - Height of tree is at least $\log_2(n!)$
- Worst-case number of comparisons performed by Alg is at least $\log_2(n!) = \Omega(n \log n)$
- Since Alg is an arbitrary comparison sort algorithm, this proves the theorem.

- Implies that Mergesort and Heapsort are "asymptotically optimal" comparison sorting algorithms
  - They run in (wc) time $O(n \log n)$
  - Must take (wc) time at least $\Omega(n \log n)$

Sorting in Linear Time

Stable sorting algorithm

- A sorting algorithm is stable if the elements with the same value (key) appear in the sorted output array in the same order in which they appear in the input array.

earlier later

e.g., [(blue,4), (purple,3), (green,5), (yellow,4)]

sort on the number

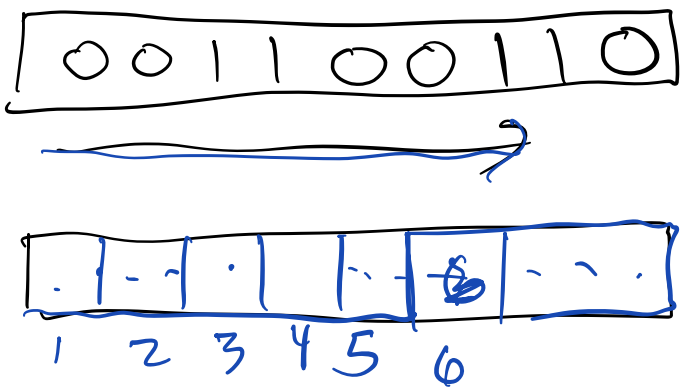
earlier later

[(blue,4), (yellow,4)

- Quicksort is not stable

Counting Sort

- Counting sort is a stable sorting algorithm used to sort an array whose elements are all integers in the range 0,1,2,…,k
- Runs in time $\theta(n + k)$
- Not an in-place algorithm
- Uses additional arrays B[1…n] and C[0…k]
  - C[0…k] is used to store the counts (number of occurrences of 0,..,k)
  - B[1…n] is the output array, the sorted version of A
- pseudocode p. 209



Count 0's

5 of them

go L to R through array

- Example:
  - k=3
  - A=[3,1,2,2,0,2]
- Step 1 of counting sort:
  - Initialize entries of C to 0
    Go through A from left to right, incrementing C[i] for each occurence of i

C[1 1 3 1]
0 1 2 3

- Example:
  - k=3
  - A=[3,1,2,2,0,2]
- Step 1 of counting sort:
  - Initialize entries of C to 0
    Go through A from left to right, incrementing C[i] for each occurrence of i

    ```
    for i=1 to k
        C[i]=0
    for j=1 to n
        C[ A[j] ] = C[ A[j] ] + 1
    ```

- Example:
  - k=3
  - A=[3,1,2,2,0,2] C=[1,1,3,1]
- Step 2 of Counting Sort: Update C so that C[i] stores number of occurrences in A of elements that are <= i

```
for i=2 to k                    C=[1,2,5,6]
    C[i] = C[i] + C[i-1]
```

- Step 3: Go through A from right to left. C[i] is the index in B where you put the next occurrence of i in A (keep C[i] updated)

- Example:
  - k=3
  - A=[3,1,2,2,0,2] C=[1,1,3,1]
- Step 2 of Counting Sort: Update C so that C[i] stores number of occurrences in A of elements that are <= i

```
for i=2 to k
    C[i] = C[i] + C[i-1]
```

- Step 3: Go through A from right to left. C[i] is the index in B where you should put the next occurrence of i in A.

```
for j = n downto 1
    B[ C[ A[j] ] ] = A[j]
    C[ A[j] ] = C[ A[j] ] - 1
```

Radix Sort

- Suppose want to sort list of d-digit numbers, where each digit is between 0 and k
- Example: d = 3, k = 9

  5 2 3
  4 2 7
  6 1 3
  2 4 5
  5 1 7
  6 1 8

- Number columns so lowest order column is 1, highest order is d
- for i=1 through d
  use counting sort to sort numbers according to column i

Radix Sort

- Example: d = 3, k = 9

  5 2 3
  4 2 7
  6 1 3
  2 4 5
  5 1 7
  6 1 8

  for i=1 through d
  use counting sort to sort numbers according to column i

- Why does this work? Loop invariant?
- Runtime?

- When computer were still very expensive, people used mechanical sorting machines to help do sorting, using principle of radix sort
  - Each number to be sorted was on a separate "punch card"
  - Knob on the machine determined the column on which to perform sort
- Today, sometimes used when sorting with multiple keys
  - e.g., want to sort by date, and date is represented by year, month, day
- When sorting n numbers, each of b-bits, can choose $r \leq b$ and use radix sort with $k = 2^r - 1$ by breaking b bits into r-bit "digits"
  - Runtime is $O((b/r)(n + 2^r))$